HPC (Lab_01)

Exercise 1: Basic Thread Parallelism

Objective: Initialize the OpenMP environment and verify multithreaded execution.

- **Task:** Write a C/C++ program that uses the #pragma omp parallel directive to print "Hello World."
- **Key Concept:** Understand the "Fork-Join" model where the master thread spawns a team of worker threads.

```cpp
#include <iostream>
#include <omp.h>
using namespace std;

int main() {

    #pragma omp parallel
    {
        cout << "Hello World\n";
    }

    return 0;
}
```

Exercise 2: Runtime Configuration and Data Environment

Objective: Learn to pass parameters to the parallel region at runtime and use OpenMP clauses.

- **Task:** Write a program that:
    1. Prompts the user to enter the number of threads at runtime.
    2. Uses the num_threads() clause to set the thread size.
    3. Prints: "Hello World from Thread [ID] of [Total Threads]."

```cpp
#include <iostream>
#include <omp.h>

using namespace std;

int main() {
    int n;
```

```
    cout << "Enter number of threads: ";
    cin >> n;

    #pragma omp parallel num_threads(n)
    {
        int id = omp_get_thread_num();
        int total = omp_get_num_threads();



        for (int i = 0; i < total; i++) {
            #pragma omp barrier
            if (i == id) {
                cout << "Hello World from Thread "
                     << id << " of " << total << endl;
            }
        }
    }

    return 0;
}
```

Exercise 3: Analyzing Race Conditions and Solution

Objective: Observe the non-deterministic nature of parallel execution and the impact of shared resources.

- **Task:** Write an OpenMP program to print "Hello World" do not ma
- **Requirement:** Introduce a manual delay (using a "wait" condition or a heavy dummy loop) inside the parallel block to force threads to overlap.
- **Observation:** Document how the output becomes scrambled (interleaved) due to multiple threads accessing the standard output simultaneously, demonstrating a **Race Condition**.
- **Solution:** Use the private() clause to ensure each thread has its own copy of the thread ID variable.

```
#include <iostream>
#include <omp.h>
using namespace std;

int main() {

    #pragma omp parallel
    {
```

```
        int id = omp_get_thread_num();

        // Dummy delay loop
        for (long long i = 0; i < 100000000; i++);

        cout << "Hello World from Thread " << id << endl;
    }

    return 0;
}


// Questions 3 -part – B ;

#include <stdio.h>
#include <omp.h>

int main() {
    int id;

    #pragma omp parallel private(id)
    {
        id = omp_get_thread_num();

        for (long long int i = 0; i < 100000000; i++);

        printf("Hello World from Thread %d\n", id);
    }

    return 0;
}
```

Exercise 4: Large-Scale Vector Addition (Data Parallelism)

Objective: Implement data decomposition on large arrays to measure parallel efficiency.

- Task: 1. Create three 1D arrays of size 1,000,000.

2. Sequentially: Initialize two arrays with random or incremental values.

3. Parallely: Use #pragma omp parallel for to calculate the sum: C[i] = A[i] + B[i].

- **Requirement:** Compare the performance of the parallel loop against a standard sequential loop.

```
#include <iostream>
```

```cpp
#include <omp.h>
using namespace std;

int main() {
    const int N = 1000000;

    int *A = new int[N];
    int *B = new int[N];
    int *C = new int[N];

    // Sequential initialization
    for (int i = 0; i < N; i++) {
        A[i] = i;
        B[i] = 2 * i;
    }

    double start = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }

    double end = omp_get_wtime();

    cout << "Parallel execution time: "
         << end - start << " seconds\n";

    // Free memory
    delete[] A;
    delete[] B;
    delete[] C;

    return 0;
}
```

Exercise 5: Parallel Reduction for Maximum Value

Objective: Use OpenMP synchronization or reduction clauses to find maximum value in a dataset.

- Task: 1. Insert values in a 1D array of size 1,000,000 with random integers sequentially.

2. Parallely: Find the maximum value in the array.

- **Requirement:** Students should implement this using the reduction(max: variable) clause to avoid manual locking while ensuring thread safety.

```cpp
#include <iostream>
#include <omp.h>
#include <cstdlib>
using namespace std;

int main() {
    const int N = 1000000;
    int arr[N];

    // Sequential initialization
    for (int i = 0; i < N; i++) {
        arr[i] = rand() % 100000;
    }

    int maxVal = arr[0];

    // here is the parallel region to find the maximum value

    #pragma omp parallel for reduction(max:maxVal)
    for (int i = 0; i < N; i++) {
        if (arr[i] > maxVal)
            maxVal = arr[i];
    }

    cout << "Maximum value: " << maxVal << endl;

    return 0;
}
```